2441 lines (2229 loc) · 95.8 KB

```
1
2     // React Theory Questions
3     // *What is React? Why use it?
4     // *What are components in React?
5     // *What is JSX?
6     // *Explain virtual DOM and how it works.
7     // *What are props and state?
8     // *What is the difference between functional and class components?
9     // *What is the useEffect Hook? How is it different from componentDidMount?
10    // *What are controlled vs uncontrolled components?
11    // *What is prop drilling? How to avoid it?
12    // *What are keys in React? Why are they important?
13    // *What is React.memo and how does it help performance?
14    // *What is the reconciliation process in React?
15    // *How does React handle re-rendering?
16    // *What is context API and when should it be used?
17    // * Explain the React lifecycle methods.
18    // *What are HOCs (Higher-Order Components)?
19    // *What is React Fiber?
20    // *What are render props?
21
22
23    // 1.   What is the Virtual DOM and how does React use it for rendering?
24    // What is the Virtual DOM?
25    // •    The DOM (Document Object Model) is the browser's tree-like structure that represen
26    // •    Directly updating the real DOM is slow, because each change forces the browser to
27    //  styles, painting, etc.).
28    // •    To solve this, React uses a Virtual DOM (VDOM):
29    // o    It's a lightweight JavaScript object that is a copy of the real DOM.
30    // o    Think of it as a "blueprint" of the UI stored in memory.
31    // ____________________________________
32    //  🔹 How React uses the Virtual DOM for Rendering
33    // 1.   Initial Render
34    // o    When you first render a React component, React creates a Virtual DOM tree.
35    // o    It then renders that structure to the real DOM only once.
36    // 2.   State/Props Update
```

```
37  // o    When data changes (state or props), React rebuilds a new Virtual DOM for that comp
38  // 3.   Diffing Algorithm (Reconciliation)
39  // o    React compares the new Virtual DOM with the previous Virtual DOM (using an efficie
40  // o    It identifies the minimal set of changes needed.
41  // 4.   Update the Real DOM
42  // o    React applies only those changes to the actual DOM (not the entire UI).
43  // o    This makes updates faster and more efficient.
44  // ____________________________________________
45  // 🔹 Benefits
46  // •    Performance: Avoids unnecessary re-rendering of the real DOM.
47  // •    Declarative UI: Developers describe what the UI should look like, and React handle
48  // •    Predictability: Makes UI updates more consistent.
49
50
51  // 2. How does React diff changes and batch updates efficiently?
52  //  React's Diffing Algorithm (Reconciliation)
53  // When state or props change:
54  // 1.   React builds a new Virtual DOM tree.
55  // 2.   It compares (diffs) the new Virtual DOM with the previous version.
56  // 3.   Instead of replacing the whole DOM, it calculates the minimum changes needed.
57  // Diffing Rules React Uses:
58  // •    Element type check:
59  // o    If the element type is the same (<div> → <div>), React reuses the existing DOM nod
60  // o    If the element type is different (<div> → <span>), React destroys the old node and
61  // •    Component diffing:
62  // o    If the component type is the same, React re-renders it with updated props/state.
63  // o    If the type changes, React discards the old component and mounts a new one.
64  // •    List diffing (Keys):
65  // o    React uses key props to efficiently match elements in arrays.
66  // o    With keys, React reuses nodes instead of re-rendering the whole list.
67  // o    Example:
68  // o    items.map(item => <li key={item.id}>{item.text}</li>)
69  // Without keys, React may destroy/recreate list items unnecessarily.
70  // ____________________________________________
71  //  Batching Updates
72  // Normally, if you run multiple setState (or state updates with Hooks), React batches the
73  // Example:
74  // setCount(count + 1);
75  // setName("Krishna");
76  // •    Instead of updating the DOM twice, React batches them and updates once after the e
77  // How React Batching Works:
78  // •    Event Handlers (default batching): React batches state updates automatically insid
79  // •    Async code (like promises, setTimeout): Before React 18, updates were not batched
80  // ____________________________________________
81  // Why This is Efficient
82  // •    Diffing ensures minimal DOM operations (DOM changes are costly).
83  // •    Batching ensures fewer re-renders (multiple state updates → one DOM update).
84  // •    Combined, React avoids unnecessary work and keeps apps fast.
85  // ____________________________________________
86  // ✅ In short:
87  // React first diffs the new Virtual DOM against the old one to find minimal changes, then
88  //  efficiently in one go.
```

```
89
90  // What happens during the React rendering and reconciliation phase?
91  // 1. Render Phase (a.k.a. Reconciliation Phase)
92  // This is the "thinking" phase 🧠. React figures out what needs to change.
93  // •    Steps:
94  // 1.   The component function (or render() in class components) is called.
95  // 2.   React builds a new Virtual DOM tree (React elements / Fiber tree).
96  // 3.   React compares the new Virtual DOM with the previous Virtual DOM (using the diffin
97  // 4.   React produces a list of changes (effects) that need to be applied to the real DOM
98  // •    Key points:
99  // o    Purely a calculation phase → no actual DOM changes yet.
100 // o    Can be paused, aborted, or restarted (with React Fiber).
101 // o    This is why React is considered asynchronous — it doesn't block the UI while diffi
102 // ____________________________________
103 // 2. Commit Phase
104 // This is the "doing" phase ⚡. React applies the changes to the real DOM.
105 // •    Steps:
106 // 1.   React takes the list of changes (diff result).
107 // 2.   It updates the real DOM nodes.
108 // 3.   Runs lifecycle methods/effects:
109 // ▯    Class components: componentDidMount, componentDidUpdate.
110 // ▯    Function components: useLayoutEffect, useEffect.
111 // •    Key points:
112 // o    This phase is synchronous (it cannot be interrupted).
113 // o    DOM updates and screen painting happen here.
114 // ____________________________________
115 // 🔹 React Fiber's Role
116 // React Fiber (introduced in React 16) is the underlying engine that:
117 // •    Splits the render/reconciliation phase into small units of work.
118 // •    Lets React pause and resume rendering for better responsiveness.
119 // •    Supports concurrent rendering (React 18+).
120 // ____________________________________
121 // 🔹 Summary (Easy Flow)
122 // 1.   Render Phase:
123 // o    Build new VDOM → Diff with old VDOM → Prepare changes.
124 // o    (Async, can be paused).
125 // 2.   Commit Phase:
126 // o    Apply changes to real DOM → Run lifecycle hooks/effects.
127 // o    (Sync, can't be paused).
128
129 // What's the role of the Fiber architecture?
130 // Why Fiber Was Introduced
131 // Before React 16, React's reconciliation worked in a synchronous stack-based way:
132 // •    When React started rendering, it would render the whole component tree recursively
133 // •    For large trees, this could block the main thread → the UI would "freeze" until Re
134 // 👉 To fix this, React introduced Fiber in React 16.
135 // ____________________________________
136 // 🔹 What is Fiber?
137 // •    A Fiber is a JavaScript object that represents a unit of work for a component.
138 // •    Think of it as a data structure that keeps track of:
139 // o    The component's type and props.
140 // o    Its state and hooks.
```

```
// o    Its place in the tree (parent/child/sibling).
// o    Side effects (what needs to happen in commit phase).
// In short: Fiber is the new reconciliation engine that breaks rendering into small piece
// ____________________________________________
// 🔹 Role of Fiber Architecture
// 1.   Interruptible Rendering
// o    Fiber allows React to pause work on one part of the tree and switch to more urgent
// o    This makes React asynchronous and non-blocking.
// 2.   Prioritization of Updates
// o    Fiber assigns priorities to updates:
// ▯    High priority: User interactions (typing, clicks).
// ▯    Low priority: Background data loading, off-screen rendering.
// o    React can work on urgent updates first, then come back to less urgent ones.
// 3.   Incremental Rendering
// o    Instead of rendering the whole tree at once, Fiber breaks it into units of work.
// o    Each Fiber node can be processed independently.
// o    If React needs to stop (say, for a user scroll), it can resume later.
// 4.   Better Scheduling
// o    React uses a cooperative scheduler: it yields control back to the browser between
// o    This keeps the app responsive.
// 5.   Foundation for Concurrent Mode (React 18+)
// o    Features like useTransition, Suspense, and concurrent rendering are possible only
// ____________________________________________
// 🔹 Fiber vs Old Reconciliation
// Feature  Old Stack Reconciler    Fiber Reconciler
// Rendering    Synchronous, recursive  Asynchronous, incremental
// Pausable?    ❌ No   ✅ Yes
// Prioritization   ❌ No   ✅ Yes
// Concurrent features  ❌ Not supported    ✅ Supported
// Data structure   Function stack  Fiber tree (linked list)
// ____________________________________________
// 🔹 Simple Analogy
// Imagine React rendering as doing chores:
// •    Old React: You must finish cleaning the entire house before answering the door.
// •    React Fiber: You can clean room by room, pause if the doorbell rings, answer it, t

// How do React hooks like useEffect and useLayoutEffect differ?
// 1. useEffect
// •    When it runs:
// After React has painted the UI to the screen (asynchronous, after the commit phase).
// •    Usage:
// For non-blocking side effects that don't affect the visible layout.
// •    Examples:
// o    Fetching data (fetch, axios).
// o    Setting up subscriptions (WebSocket, event listeners).
// o    Logging, analytics.
// •    Why:
// Since it runs after painting, it doesn't block the browser from updating the UI → keeps
// useEffect(() => {
//   console.log("Runs after paint");
//   document.title = `Count: ${count}`;
// }, [count]);
```

```
// ____________________________________________
// 🔹 2. useLayoutEffect
// •    When it runs:
// Immediately after React updates the DOM but before the browser paints the screen (synch
// •    Usage:
// For synchronous side effects that must run before the user sees the update.
// •    Examples:
// o    Measuring DOM nodes (getBoundingClientRect).
// o    Synchronously mutating styles or scroll position.
// o    Animations that need precise initial measurements.
// •    Why:
// Since it runs before paint, the user won't see flickers (layout shifts).
// useLayoutEffect(() => {
//   console.log("Runs before paint");
//   const rect = divRef.current.getBoundingClientRect();
//   console.log("Element size:", rect);
// }, []);
// ____________________________________________
// 🔹 Key Differences
// Feature  useEffect 🕐    useLayoutEffect ⚡
// Timing   After paint (async) Before paint (sync)
// Blocks visual updates?  ❌ No  ✅ Yes
// Use case Non-UI side effects (data, logs, subscriptions) DOM measurements, style/layout
// Performance  More performant, doesn't block paint    Can cause jank if overused
// ____________________________________________
// 🔹 Rule of Thumb
// •    ✅ Use useEffect by default.
// •    ✅ Use useLayoutEffect only when you need to measure or modify DOM layout before t
// •    🚫 Don't overuse useLayoutEffect → it can block painting and hurt performance.

// What are controlled vs uncontrolled components in forms?
//  → When would you use each, and why?
// 🔹 Controlled Components
// •    A controlled component is a form element (like <input>, <textarea>, <select>) whos
// •    The React component is the "single source of truth".
// Example:
// function ControlledInput() {
//   const [name, setName] = React.useState("");

//   return (
//     <input
//       type="text"
//       value={name}
//       onChange={(e) => setName(e.target.value)}
//     />
//   );
// }
// •    Here:
// o    value={name} → React decides what's shown.
// o    Every keystroke triggers onChange, which updates state → rerenders → updates input
// ✅ Pros:
// •    Easy to validate, transform, or restrict input in real-time.
```

```
// •    Keeps form data centralized in React state → predictable.
// •    Works well for complex forms (with validation, conditional logic).
// ❌ Cons:
// •    More boilerplate code (handlers, state).
// •    Slightly more overhead for large forms.
// ____________________________________
// 🔹 Uncontrolled Components
// •    An uncontrolled component is a form element that manages its own state internally
// •    React doesn't control the value directly. Instead, you access it using a ref when
// Example:
// function UncontrolledInput() {
//   const inputRef = React.useRef();

//   function handleSubmit() {
//     alert(inputRef.current.value); // Read directly from DOM
//   }

//   return (
//     <>
//       <input type="text" ref={inputRef} />
//       <button onClick={handleSubmit}>Submit</button>
//     </>
//   );
// }
// •    Here:
// o    Input value is stored in the DOM.
// o    React only grabs it when needed via ref.
// ✅ Pros:
// •    Less code, simpler setup for small forms.
// •    Useful for quick uncontrolled data entry (like search boxes).
// •    Can integrate with non-React code (jQuery, legacy scripts).
// ❌ Cons:
// •    Harder to enforce validation or formatting while typing.
// •    Multiple sources of truth (DOM + React state).
// •    Not ideal for complex, dynamic forms.
// ____________________________________
// 🔹 When to Use Each?
// Use Controlled Components when:
// •    You need real-time validation (e.g., password strength meter).
// •    You want to conditionally enable/disable UI based on input.
// •    You need to save form data in React state for later use (e.g., multi-step forms, w
// Use Uncontrolled Components when:
// •    You just need the value once on submit, not on every keystroke.
// •    You're working with simple forms or migrating from legacy DOM code.
// •    You want less boilerplate and don't care about React tracking every keystroke.

// What causes unnecessary re-renders in React and how can you optimize them?
//  → How can React.memo, useMemo, and useCallback help?
// What Causes Unnecessary Re-renders in React?
// React re-renders a component when:
// 1.   Parent component re-renders (children re-render unless optimized).
// 2.   Props change (even if the new value is referentially different but logically the s
```

```
297  // 3.   <Child data={{ name: "Krishna" }} /> // new object every render
298  // 4.   State updates (even if setting the same value again).
299  // 5.   Context updates (all consumers re-render on context value change).
300  // 6.   Anonymous functions / inline objects inside JSX (new reference each render).
301  // ____________________________________
302  // 🔹 How to Optimize Re-renders?
303  // 1. React.memo
304  // •    Wraps a functional component.
305  // •    Prevents re-rendering if props haven't changed (shallow comparison).
306  // const Child = React.memo(({ name }) => {
307  //   console.log("Child rendered");
308  //   return <div>{name}</div>;
309  // });
310
311  // function Parent() {
312  //   const [count, setCount] = React.useState(0);
313  //   return (
314  //     <>
315  //       <button onClick={() => setCount(count + 1)}>+</button>
316  //       <Child name="Krishna" /> {/* Won't re-render unnecessarily */}
317  //     </>
318  //   );
319  // }
320  // ✅ Best for pure components where props rarely change.
321  // ⚠️ Doesn't help if you pass new objects/functions every render.
322  // ____________________________________
323  // 2. useMemo
324  // •    Memoizes the result of a computation.
325  // •    Prevents recalculating expensive values on every render.
326  // const filteredList = useMemo(() => {
327  //   return items.filter(item => item.active);
328  // }, [items]);
329  // ✅ Best for expensive calculations or derived data.
330  // ⚠️ Don't overuse — memoization itself has overhead.
331  // ____________________________________
332  // 3. useCallback
333  // •    Memoizes a function definition.
334  // •    Prevents creating a new function reference on every render.
335  // const handleClick = useCallback(() => {
336  //   console.log("Clicked");
337  // }, []); // function reference stays stable
338  // •    Without useCallback, passing functions to children causes re-renders (because a ne
339  // •    With useCallback, child components wrapped in React.memo can skip re-renders.
340  // ✅ Best when passing callbacks to memoized children.
341  // ____________________________________
342  // 🔹 Quick Comparison
343  // Hook / Tool  Prevents    Best Use Case
344  // React.memo   Child re-renders due to unchanged props Pure components
345  // useMemo  Re-computing expensive values   Derived data, filtering, sorting
346  // useCallback  Re-creating functions on each render    Passing stable callbacks to childr
347  // ____________________________________
348  // 🔹 Other Optimization Techniques
```

```
// •    Key usage in lists → Use stable key to avoid unnecessary re-mounts.
// •    Split large components → Smaller components re-render less often.
// •    Context optimization → Split contexts or use libraries like zustand or jotai for f
// •    Windowing/Lazy rendering → For big lists, use libraries like react-window.

// How does context API compare to Redux for state management?
//  → When is one preferable over the other?
// 1. React Context API
// •    What it is:
// A built-in React feature for prop drilling avoidance. Lets you share data globally (the
// •    How it works:
// o    Create a context → Wrap part of your app with a Provider → Any component inside ca
// const ThemeContext = React.createContext("light");

// function App() {
//   return (
//     <ThemeContext.Provider value="dark">
//       <Child />
//     </ThemeContext.Provider>
//   );
// }

// function Child() {
//   const theme = React.useContext(ThemeContext);
//   return <p>Theme: {theme}</p>;
// }
// •    Pros:
// o    Simple, built-in (no extra library).
// o    Great for small/medium apps.
// o    Perfect for static or low-frequency data (theme, locale, auth).
// •    Cons:
// o    Every context update re-renders all consumers (can cause performance issues in lar
// o    No built-in devtools for debugging.
// o    No opinionated structure (easy to misuse for complex state).
// ____________________________________
//  🔹 2. Redux
// •    What it is:
// A state management library for predictable global state.
// Uses a single store, actions, and reducers to manage state updates.
// •    How it works:
// o    You dispatch an action → Reducer updates the state → Components subscribed to that
// slice with Redux Toolkit
// const counterSlice = createSlice({
//   name: "counter",
//   initialState: 0,
//   reducers: {
//     increment: state => state + 1,
//     decrement: state => state - 1
//   }
// });

// const { increment } = counterSlice.actions;
```

```
// •    Pros:
// o    Scales well for large/complex apps.
// o    Predictable: pure reducers, strict flow.
// o    Excellent devtools (time travel debugging, logging).
// o    Ecosystem support: middleware (redux-thunk, redux-saga), persistence, etc.
// o    Fine-grained re-render control (connect, selectors).
// •    Cons:
// o    More boilerplate than Context (though Redux Toolkit reduced this a lot).
// o    Learning curve for beginners.
// o    Can be "overkill" for small apps.
// ____________________________________________
// 🔹 When to Use Which?
// ✅ Prefer Context API when:
// •    Your app is small/medium.
// •    You just need to avoid prop drilling.
// •    Global state updates are infrequent (theme, auth, language).
// •    You want zero extra dependencies.
// ✅ Prefer Redux when:
// •    Your app is large, complex, or collaborative.
// •    You need predictable state flow with debugging tools.
// •    You have frequent or complex state updates (e.g., forms, API data, caching, undo/r
// •    You want middleware support (logging, async actions, side effects).
// •    You need to optimize performance (Redux selectors prevent unnecessary re-renders).
// ____________________________________________
// 🔹 Quick Comparison
// Feature  Context API 🟦  Redux 🔴
// Built-in?     ✅ Yes  ❌ No (external lib)
// Setup Complexity Low Medium/High
// Debugging Tools  ❌ Limited  ✅ Powerful DevTools
// Best for Small apps, simple global state Large apps, complex state logic
// Performance on updates   Re-renders all consumers    Fine-grained control via selectors
// Async logic handling Manual (useEffect)  Middleware (redux-thunk, saga, etc.)

// What is prop drilling and how do you avoid it?
//  → What design patterns help in large applications?

// What is Prop Drilling?
// •    Prop drilling happens when you pass props through multiple layers of components ju
// •    The intermediate components don’t care about the data but still have to pass it do
// Example:
// function App() {
//   const user = { name: "Krishna" };
//   return <Parent user={user} />;
// }

// function Parent({ user }) {
//   return <Child user={user} />;
// }

// function Child({ user }) {
//   return <GrandChild user={user} />;
// }
```

```
// function GrandChild({ user }) {
//   return <p>Hello {user.name}</p>;
// }
// 👉 Here, user prop is drilled through Parent → Child → GrandChild even though only Grar
// ____________________________________
// 🔹 Why is Prop Drilling a Problem?
```

main Last_Believe / react.js ↑ Top

Code Blame Raw

```
// 🔹 How to Avoid Prop Drilling?
// 1. React Context API (Built-in)
// •    Provides a way to pass data directly to any component in the tree without prop dri
// const UserContext = React.createContext();

// function App() {
//   const user = { name: "Krishna" };
//   return (
//     <UserContext.Provider value={user}>
//       <GrandChild />
//     </UserContext.Provider>
//   );
// }

// function GrandChild() {
//   const user = React.useContext(UserContext);
//   return <p>Hello {user.name}</p>;
// }
// ✅ Great for themes, auth, language.
// ⚠️ Not ideal for very frequent updates (causes re-renders).
// ____________________________________
// 2. State Management Libraries (Redux, Zustand, Jotai, Recoil)
// •    Provide a global store so components can read/write state without drilling props.
// •    Useful when you have complex app state (caching, multiple data sources, async upda
// ____________________________________
// 3. Component Composition
// •    Instead of passing props down many layers, pass children as functions/components.
// function Layout({ header, content }) {
//   return (
//     <div>
//       <header>{header}</header>
//       <main>{content}</main>
//     </div>
//   );
// }

// function App() {
//   return (
//     <Layout
//       header={<Header />}
```

```
505  //       content={<Dashboard />}
506  //     />
507  //   );
508  // }
509  // ✅ Reduces unnecessary props.
510  // ____________________________________________
511  // 4. Custom Hooks
512  // •    Extract shared logic into hooks so you don’t need to pass props everywhere.
513  // function useUser() {
514  //   return React.useContext(UserContext);
515  // }
516
517  // function Profile() {
518  //   const user = useUser();
519  //   return <p>{user.name}</p>;
520  // }
521  // ____________________________________________
522  // 5. Event Emitters / Pub-Sub Pattern
523  // •    Useful for loosely coupled communication between components.
524  // •    Instead of drilling, a child can emit an event, and another component can listen.
525  // •    Often replaced by context or state management libs in React.
526  // ____________________________________________
527  // 🔹 Design Patterns for Large Applications
528  // •    Context for static/global values (auth, theme, locale).
529  // •    Redux/Zustand/Recoil for complex state with frequent updates.
530  // •    Compound Components Pattern (pass only what’s needed via composition).
531  // •    Custom Hooks for reusing business logic.
532  // •    Colocation (keep state as close as possible to where it’s needed to avoid lifting
533
534  // How do keys work in lists and why are they important?
535  // //  → What happens if keys are not unique?
536  // 🔹 What are Keys in Lists?
537  // •    A key is a special prop React uses to identify which items in a list have changed,
538  // •    It helps React’s reconciliation algorithm (diffing) decide how to update the DOM e
539  // Example:
540  // const items = ["A", "B", "C"];
541
542  // function MyList() {
543  //   return (
544  //     <ul>
545  //       {items.map((item, index) => (
546  //         <li key={item}>{item}</li>
547  //       ))}
548  //     </ul>
549  //   );
550  // }
551  // Here, each <li> has a key (A, B, C).
552  // ____________________________________________
553  // 🔹 Why are Keys Important?
554  // Without keys, React has to re-render everything blindly because it can’t track which it
555  // With keys:
556  // •    React reuses DOM nodes for unchanged items.
```

```
// •    Only creates/removes/updates the nodes that changed.
// •    Improves performance and prevents UI glitches.
// ____________________________________
// 🔹 What Happens if Keys Are Not Unique?
// If keys are duplicated or unstable:
// 1.   Incorrect UI updates – React may reuse the wrong component instance.
// o    Example: input values resetting when list changes.
// 2.   Performance loss – React may re-render unnecessarily.
// 3.   Bugs in stateful components – A component might keep the wrong state because React
// Example Problem:
// const items = ["Apple", "Banana", "Cherry"];

// function MyList() {
//   return (
//     <ul>
//       {items.map((item, index) => (
//         <li key={index}>{item}</li>  // ❌ Bad: using index
//       ))}
//     </ul>
//   );
// }
// 👉 If we reorder items (["Banana", "Apple", "Cherry"]), React thinks only the text char
// ____________________________________
// 🔹 Best Practices for Keys
// ✅ Use a unique, stable identifier (like id from DB).
// ✅ Use index only when:
// •    List is static (no reordering/deletion).
// •    Items don't have unique IDs.
// ❌ Don't use random values (e.g., Math.random()), as they change on every render.

//  How do custom hooks work and what are their benefits?
// //  → Have you created a reusable hook in a project?
// What are Custom Hooks?
// •    A custom hook is a JavaScript function whose name starts with use and can call oth
// •    They let you extract reusable logic from components so it can be shared across mul
// Example: A simple custom hook
// import { useState, useEffect } from "react";

// function useWindowWidth() {
//   const [width, setWidth] = useState(window.innerWidth);

//   useEffect(() => {
//     const handleResize = () => setWidth(window.innerWidth);
//     window.addEventListener("resize", handleResize);
//     return () => window.removeEventListener("resize", handleResize);
//   }, []);

//   return width;
// }

// // Usage in a component
// function MyComponent() {
```

```
609     //   const width = useWindowWidth();
610     //   return <p>Window width: {width}px</p>;
611     // }
612     // ✅ Here:
613     // •    The logic for tracking window width is encapsulated in useWindowWidth.
614     // •    Any component can reuse it without rewriting the logic.
615     // ____________________________________________
616     // 🔹 Benefits of Custom Hooks
617     // 1.   Reusability – Write logic once, use it anywhere.
618     // 2.   Separation of concerns – Keeps components clean; hooks handle the logic.
619     // 3.   Testability – Easier to test logic separately from UI.
620     // 4.   Readability & maintainability – Avoid repeating code in multiple components.
621     // ____________________________________________
622     // 🔹 Common Use Cases
623     // •    API calls / data fetching
624     // •    Form input handling and validation
625     // •    Local storage management
626     // •    Window size or scroll position tracking
627     // •    Authentication/session management
628     // ____________________________________________
629     // 🔹 Real-world Example (from a project)
630     // Suppose I built a finance tracker app:
631     // •    I created a custom hook useFetchData(url) to fetch transactions from API:
632     // function useFetchData(url) {
633     //   const [data, setData] = useState([]);
634     //   const [loading, setLoading] = useState(true);
635     //
636     //   useEffect(() => {
637     //     async function fetchData() {
638     //       setLoading(true);
639     //       const res = await fetch(url);
640     //       const result = await res.json();
641     //       setData(result);
642     //       setLoading(false);
643     //     }
644     //     fetchData();
645     //   }, [url]);
646     //
647     //   return { data, loading };
648     // }
649     //
650     // // Usage
651     // function Transactions() {
652     //   const { data, loading } = useFetchData("/api/transactions");
653     //   if (loading) return <p>Loading...</p>;
654     //   return <ul>{data.map(t => <li key={t.id}>{t.amount}</li>)}</ul>;
655     // }
656     // •    Benefit: Any component that needs API data can just use useFetchData(url) without
657     //
658     // What is React’s Concurrent Mode and how does it improve UX?
659     // //  → What are transitions and how do they relate to concurrent features?
660     // 1. What is Concurrent Mode?
```

```
// •    Concurrent Mode is a set of React features (React 18+) that allows React to interr
// •    Goal: Make apps feel more responsive, especially under heavy updates.
// Key Idea:
// •    Traditional React is synchronous: once rendering starts, it blocks the UI until co
// •    Concurrent Mode is asynchronous: rendering can be paused, delayed, or interrupted
// _____________________________________
// 🔹 2. How Concurrent Mode Improves UX
// •    Non-blocking rendering: User interactions (clicks, typing) stay responsive even wh
// •    Smooth transitions: React can delay non-urgent updates without freezing the interf
// •    Avoids jank: Heavy computations or data fetching won't block the UI.
// _____________________________________
// 🔹 3. Transitions
// •    Transitions are a new API in Concurrent Mode to mark certain state updates as "non
// •    Example: Updating a search results list while keeping the search input responsive.
// API:
// const [search, setSearch] = useState("");
// const [results, setResults] = useState([]);
// const [isPending, startTransition] = useTransition();

// function handleChange(e) {
//   setSearch(e.target.value); // urgent update (input)
//   startTransition(() => {
//     // non-urgent update (results list)
//     setResults(filterData(e.target.value));
//   });
// }
// How it works:
// 1.   setSearch → urgent → input updates immediately.
// 2.   startTransition → non-urgent → list update can be deferred if the browser is busy.
// 3.   React keeps the UI interactive while rendering the heavy part in the background.
// ✅ You can also use isPending to show a loading indicator during non-urgent updates.
// _____________________________________
// 🔹 4. Relation Between Concurrent Mode and Transitions
// Feature  Role in Concurrent Mode
// Concurrent Mode  Makes React rendering interruptible & non-blocking
// Transitions  Marks certain updates as low-priority to avoid UI blocking
// •    Together, they allow responsive apps even under heavy computation or large UI tree
// _____________________________________
// 🔹 5. Example Use Cases
// •    Search/filter UI: input stays responsive while results update.
// •    Pagination or virtualized lists: scrolling stays smooth.
// •    Animations: can render complex UI without jank.
// •    Large forms: typing isn't blocked while validating or saving data.

// What is hydration in SSR and how does it work with React?
// //  → What issues have you faced with SSR/Next.js?
// 1. What is Hydration?
// •    Hydration is the process where a server-rendered HTML page is "attached" to the Re
// •    SSR sends fully rendered HTML to the browser, but without JavaScript, it's static
// •    React then hydrates the DOM, attaches event listeners, and makes the page dynamic.
// How it works:
// 1.   Server-side: React renders components to HTML and sends it to the browser.
```

```
// 2.   <button>Click me</button> <!-- HTML already rendered -->
// 3.   Client-side (hydration): React runs JS, attaches event listeners, and manages stat
// 4.   const [count, setCount] = useState(0); // now interactive
// 5.   <button onClick={() => setCount(count + 1)}>Click me</button>
// •    After hydration, the page behaves like a normal React SPA.
// ____________________________________________
// 🔹 2. Why Hydration is Important
// •    Improves initial page load → faster content for SEO and users.
// •    Makes the page interactive without re-rendering the entire DOM.
// •    Allows SSR + CSR hybrid: static content for speed, React for interactivity.
// ____________________________________________
// 🔹 3. Common Issues Faced with SSR / Hydration
// 1. Mismatch Between Server & Client
// •    React compares the server HTML and client-rendered HTML.
// •    If they differ, you get warnings:
// •    Warning: Text content did not match. Server: "X" Client: "Y"
// •    Causes:
// o    Using browser-specific APIs (window, document) on the server.
// o    Random values (Math.random, Date.now) during render.
// o    Non-deterministic rendering.
// 2. Event Listeners Not Attached
// •    SSR renders HTML, but without hydration, buttons/inputs aren’t interactive.
// 3. Performance Overhead
// •    Large pages may take time to hydrate, causing “interactive delay”.
// •    Heavy JS bundles can slow down hydration.
// 4. State Initialization Issues
// •    Server has no access to client state (localStorage, sessionStorage).
// •    You may need useEffect to initialize state after hydration.
// ____________________________________________
// 🔹 4. How to Handle These Issues
// 1.   Conditional client-only code:
// 2.   useEffect(() => {
// 3.     console.log(window.innerWidth);
// 4.   }, []);
// 5.   Avoid non-deterministic server-side rendering (e.g., no Math.random() in render).
// 6.   Use Next.js dynamic imports for client-only components:
// 7.   import dynamic from 'next/dynamic';
// 8.   const Chart = dynamic(() => import('./Chart'), { ssr: false });
// 9.   Split hydration: defer heavy components using React.lazy or Suspense.
// 10.  Check hydration warnings: make sure server & client markup match exactly.

// How does error handling work in React with Error Boundaries?
// //  → What types of errors can and can’t be caught?
// 1. What Are Error Boundaries?
// •    Error boundaries are React components that catch JavaScript errors anywhere in the
// •    They prevent the whole React component tree from crashing and let you show a fallb
// How to create one (class component):
// class ErrorBoundary extends React.Component {
//   constructor(props) {
//     super(props);
//     this.state = { hasError: false };
//   }
```

```
765
766     //   static getDerivedStateFromError(error) {
767     //     // Update state to show fallback UI
768     //     return { hasError: true };
769     //   }
770
771     //   componentDidCatch(error, info) {
772     //     // Log error to an external service
773     //     console.log(error, info);
774     //   }
775
776     //   render() {
777     //     if (this.state.hasError) {
778     //       return <h2>Something went wrong.</h2>;
779     //     }
780     //     return this.props.children;
781     //   }
782     // }
783
784     // // Usage
785     // function App() {
786     //   return (
787     //     <ErrorBoundary>
788     //       <MyComponent />
789     //     </ErrorBoundary>
790     //   );
791     // }
792     // ____________________________________________
793     //  🔹 2. What Types of Errors Can Be Caught?
794     // Error boundaries can catch errors in:
795     // 1.   Render phase – errors during rendering of child components.
796     // 2.   Lifecycle methods – errors in componentDidMount, componentDidUpdate.
797     // 3.   Constructor of child components – errors in child class component constructors.
798     // ____________________________________________
799     //  🔹 3. What Cannot Be Caught?
800     // Error boundaries cannot catch:
801     // 1.   Event handlers – handle separately with try/catch.
802     // 2.   <button onClick={() => { throw new Error("Oops") }}>Click</button>
803     // o    React does not crash the whole app; the error must be handled manually.
804     // 3.   Asynchronous code – e.g., setTimeout, fetch promises.
805     // 4.   Server-side rendering errors – errors during SSR must be handled differently.
806     // 5.   Errors in the error boundary itself – must wrap multiple boundaries if needed.
807     // ____________________________________________
808     //  🔹 4. Using Hooks with Error Boundaries
809     // •    You cannot use a functional component with hooks alone as an error boundary.
810     // •    Only class components can be error boundaries.
811     // •    Workarounds: use a wrapper class boundary to catch errors for functional children.
812     // ____________________________________________
813     //  🔹 5. Best Practices
814     // •    Place error boundaries around sections of the app, not just the root.
815     // •    Use fallback UI that allows the app to continue running.
816     // •    Log errors to monitoring services (Sentry, LogRocket).
```

```
817  // •    Combine with try/catch in async operations or event handlers.
818
819  // What is the difference between useRef and createRef?
820  // //  → When should refs be avoided altogether?
821
822  // 1. useRef (Hook, functional components)
823  // •    Purpose: Persist a mutable value across renders without triggering re-renders.
824  // •    Scope: Works inside functional components.
825  // •    Returns an object: { current: ... }
826  // Example:
827  // import { useRef, useEffect } from "react";
828
829  // function Timer() {
830  //   const intervalRef = useRef(null);
831
832  //   useEffect(() => {
833  //     intervalRef.current = setInterval(() => console.log("tick"), 1000);
834  //     return () => clearInterval(intervalRef.current);
835  //   }, []);
836
837  //   return <p>Timer running in console</p>;
838  // }
839  // ✅ Key points:
840  // •    intervalRef.current persists across renders.
841  // •    Updating current does not cause a re-render.
842  // ____________________________________
843  // 🔹 2. createRef (Class components)
844  // •    Purpose: Access a DOM node or a React component instance.
845  // •    Scope: Typically used in class components.
846  // •    Returns a new ref on each render, so it’s usually attached to instance variables i
847  // Example:
848  // class MyInput extends React.Component {
849  //   constructor(props) {
850  //     super(props);
851  //     this.inputRef = React.createRef();
852  //   }
853
854  //   focusInput = () => {
855  //     this.inputRef.current.focus();
856  //   }
857
858  //   render() {
859  //     return <input ref={this.inputRef} />;
860  //   }
861  // }
862  // ✅ Key points:
863  // •    createRef does not persist across functional component renders.
864  // •    Works best in class components.
865  // ____________________________________
866  // 🔹 3. Key Differences
867  // Feature  useRef (functional) createRef (class)
868  // Component type   Functional  Class
```

```
// Persistence  Same ref across renders New ref every render
// Updates trigger rerender?     ❌ No   ❌ No
// Common use cases DOM access, timers, mutable values  DOM access, child component instan
// ____________________________________________
// 🔹 4. When to Avoid Refs
// Refs should be used sparingly. Prefer state/props instead. Avoid refs when:
// 1.   Controlling UI state – e.g., form inputs should use controlled components, not ref
// 2.   Conditional rendering – rely on state, not refs, for showing/hiding components.
// 3.   Complex logic – keep data in state to leverage React's reconciliation and re-rende
// ✅ Rule of Thumb: "Refs are escape hatches, not replacements for state."
// ____________________________________________
// 🔹 5. When Refs Are Useful
// •    Accessing DOM elements (focus, scroll, measure).
// •    Storing mutable values that don't trigger re-renders (timers, intervals, previous
// •    Integrating with third-party libraries that require DOM nodes.

// How does React handle synthetic events under the hood?
// //  → What are the performance benefits of the synthetic event system?
// 1. What Are Synthetic Events?
// •    React wraps native browser events in its own SyntheticEvent object.
// •    This provides a consistent API across all browsers (cross-browser compatibility).
// •    Works the same for all events: click, change, keypress, etc.
// function Button() {
//   const handleClick = (event) => {
//     console.log(event.type); // "click"
//   };

//   return <button onClick={handleClick}>Click Me</button>;
// }
// •    Here, event is a SyntheticEvent, not the raw DOM event.
// ____________________________________________
// 🔹 2. How It Works Under the Hood
// 1.   Single Event Listener
// o    React attaches a single event listener at the root of the DOM (event delegation).
// o    Example: React sets one listener on document instead of adding one per element.
// 2.   Event Pooling (in older versions)
// o    Synthetic events were pooled for memory efficiency (React reuses event objects).
// o    In React 17+, pooling is removed, but the synthetic wrapper still exists.
// 3.   Propagation Control
// o    React normalizes event propagation (stopPropagation, preventDefault) across browse
// ____________________________________________
// 🔹 3. Performance Benefits
// 1.   Memory Efficiency
// o    Instead of attaching many listeners for every component, React uses one listener p
// 2.   Faster Event Handling
// o    Centralized delegation avoids multiple DOM lookups.
// o    Updates can be batched efficiently during the event.
// 3.   Cross-Browser Consistency
// o    You don't have to worry about differences in addEventListener, event bubbling, or
// 4.   Simplified Cleanup
// o    No need to manually remove listeners in most cases; React handles it automatically
// ____________________________________________
```

```
// 🔹 4. Things to Note
// •    Synthetic events are pooled in older versions, so you must call event.persist() if
// •    React's synthetic events still use the native events under the hood; the wrapper j
// •    Works seamlessly with hooks, state updates, and batching in modern React.

// You have a large form with many inputs.how would you manage and optimize state?
// 1. Controlled vs Uncontrolled Inputs
// •    Controlled inputs: state lives in React → easy validation, dynamic UI updates.
// •    Uncontrolled inputs: state lives in DOM → less re-rendering, simpler for static fo
// Recommendation: For large, dynamic forms, use controlled components for critical fields
// ____________________________________________
// 🔹 2. State Management Strategies
// a) Single State Object
// •    Store all form values in a single object using useState:
// const [formData, setFormData] = useState({
//   firstName: "",
//   lastName: "",
//   email: "",
//   // ...more fields
// });

// function handleChange(e) {
//   const { name, value } = e.target;
//   setFormData(prev => ({ ...prev, [name]: value }));
// }
// ✅ Pros:
// •    Easy to submit the entire form.
// •    Centralized state.
// ⚠️ Cons:
// •    Every change re-renders the entire form, which can hurt performance for large form
// ____________________________________________
// b) Split State per Input / Field
// const [firstName, setFirstName] = useState("");
// const [lastName, setLastName] = useState("");
// ✅ Pros:
// •    Updates only the relevant input → fewer re-renders.
// ⚠️ Cons:
// •    Becomes verbose with many fields.
// ____________________________________________
// c) useReducer
// •    For complex forms with nested fields or dynamic input arrays:
// function reducer(state, action) {
//   switch(action.type) {
//     case "UPDATE_FIELD":
//       return { ...state, [action.field]: action.value };
//     default: return state;
//   }
// }

// const [formState, dispatch] = useReducer(reducer, initialState);

// <input
```

```
//   name="firstName"
//   value={formState.firstName}
//   onChange={(e) => dispatch({ type: "UPDATE_FIELD", field: e.target.name, value: e.targ
// />
// ✅ Pros:
// •    Centralized logic for validation, updates, and resets.
// •    Great for dynamic/nested forms.
// _____________________________________
// 🔹 3. Performance Optimization
// 1. React.memo
// •    Wrap form sections or input components with React.memo to prevent unnecessary re-r
// 2. useCallback
// •    Memoize handlers to prevent child re-rendering:
// const handleChange = useCallback((e) => {
//   dispatch({ type: "UPDATE_FIELD", field: e.target.name, value: e.target.value });
// }, []);
// 3. Lazy Rendering / Virtualization
// •    For very long forms (100+ fields), render only visible fields, similar to virtuali
// 4. Debounce Expensive Computations
// •    For live validation, API calls, or formatting, use debounce to reduce frequency of
// _____________________________________
// 🔹 4. Libraries That Help
// For very large forms, consider libraries designed for performance and scalability:
// •    Formik – popular and battle-tested.
// •    React Hook Form – minimal re-renders, very performant for large forms.
// •    Final Form – provides advanced control over state and validation.
// Example: React Hook Form only re-renders inputs that change, making it ideal for large
// _____________________________________
// 🔹 5. Best Practices Summary
// 1.   Use controlled components for important inputs.
// 2.   Group fields using useReducer or libraries like React Hook Form.
// 3.   Optimize re-renders with React.memo and useCallback.
// 4.   Avoid updating state unnecessarily (don't copy entire state if only one field chan
// 5.   Use lazy rendering or virtualization for extremely large forms.

// You're seeing unnecessary re-renders in a component how do you debug and fix it?
// 1. Identify the Problem
// a) React DevTools Profiler
// •    Open React DevTools → Profiler.
// •    Record interactions and check which components are re-rendering frequently.
// •    Look for:
// o    Components updating even when props/state haven't changed.
// o    Large components re-rendering unnecessarily.
// _____________________________________
// b) Logging Render
// function MyComponent(props) {
//   console.log("MyComponent rendered");
//   return <div>{props.value}</div>;
// }
// •    Logs in console help identify how often a component renders.
// •    Combine with Profiler for more insights.
// _____________________________________
```

```
// 🔹 2. Common Causes of Unnecessary Re-renders
// 1.   Parent Re-rendering
// o    Even if the child's props didn't change, it re-renders by default.
// 2.   Inline Functions / Objects / Arrays
// 3.   <Child onClick={() => doSomething()} />   // new function every render
// 4.   <Child data={{ a: 1 }} />                // new object every render
// 5.   State updates that don't actually change state
// 6.   setCount(count); // triggers a re-render even if value didn't change
// 7.   Context updates
// o    Any change in context value triggers all consumers to re-render.
// ____________________________________________
// 🔹 3. How to Fix / Optimize
// a) React.memo for Functional Components
// •    Memoize the component to skip re-render if props didn't change.
// const Child = React.memo(function Child({ value }) {
//   console.log("Child rendered");
//   return <div>{value}</div>;
// });
// b) useCallback for Functions
// •    Memoize functions passed as props to avoid new references on every render.
// const handleClick = useCallback(() => doSomething(), []);
// <Child onClick={handleClick} />
// c) useMemo for Expensive Calculations
// •    Memoize derived data so it doesn't recompute every render.
// const processedData = useMemo(() => expensiveCalculation(data), [data]);
// d) Split State / Lift State Wisely
// •    Avoid putting unrelated state in one parent.
// •    Split state to local components to minimize re-renders.
// e) Optimize Context Usage
// •    Only wrap components that need context.
// •    Consider splitting context for different pieces of state.
// f) Avoid Inline Objects / Arrays in JSX
// // Bad
// <Child data={{ a: 1 }} />

// // Good
// const data = useMemo(() => ({ a: 1 }), []);
// <Child data={data} />
// ____________________________________________
// 🔹 4. Tools to Aid Debugging
// •    React DevTools Profiler – visualize re-renders.
// •    why-did-you-render – NPM package that logs unnecessary re-renders in development.
// •    console.log – simple but effective for small components.
// ____________________________________________
// 🔹 5. Best Practices Summary
// 1.   Use React.memo for pure functional components.
// 2.   Memoize functions and objects passed as props with useCallback / useMemo.
// 3.   Avoid putting unrelated state in the same component.
// 4.   Minimize context consumers to only components that need them.
// 5.   Use Profiler to measure and verify performance improvements.

// A parent component re-renders and causes all children to re-render.how can you optimize
```

```
// 1. Why Children Re-render
// •    In React, when a parent re-renders, all its children re-render by default.
// •    Even if child props haven’t changed, the child function runs again.
// •    Causes unnecessary computations and can hurt performance in large trees.
// ____________________________________________
// 🔹 2. Optimization Strategies
// a) React.memo
// •    Wrap child components with React.memo to prevent re-render if props didn’t change.
// const Child = React.memo(({ value }) => {
//   console.log("Child rendered");
//   return <div>{value}</div>;
// });

// function Parent() {
//   const [count, setCount] = useState(0);
//   return (
//     <>
//       <button onClick={() => setCount(count + 1)}>Increment</button>
//       <Child value="Static Data" />
//     </>
//   );
// }
// ✅ Now Child only re-renders if value changes.
// ____________________________________________
// b) useCallback for Functions
// •    If a parent passes inline functions as props, they are recreated each render → chi
// const handleClick = useCallback(() => doSomething(), []);
// <Child onClick={handleClick} />
// •    useCallback ensures the function reference stays stable.
// ____________________________________________
// c) useMemo for Objects / Arrays
// •    Passing inline objects/arrays also triggers child re-renders.
// // Bad
// <Child data={{ a: 1 }} />

// // Good
// const data = useMemo(() => ({ a: 1 }), []);
// <Child data={data} />
// ____________________________________________
// d) Split State
// •    Keep state local to the component that needs it instead of storing it in the paren
// •    This reduces the frequency of parent re-renders.
// function Parent() {
//   const [count, setCount] = useState(0);
//   return <Child />;
// }
// •    Child won’t re-render if it doesn’t depend on count.
// ____________________________________________
// e) Optimize Context Usage
// •    If a parent re-renders due to context updates, all context consumers re-render.
// •    Split context into multiple providers for unrelated pieces of state.
// ____________________________________________
```

```
// f) Lazy / Dynamic Rendering
// •    For large child lists, consider:
// o    React.lazy / Suspense for dynamic imports.
// o    Windowing / virtualization for long lists (react-window, react-virtualized).
// ____________________________________
// 🔹 3. Summary of Techniques
// Problem Cause    Solution
// Child re-renders with same props React.memo
// Inline functions as props    useCallback
// Inline objects/arrays as props   useMemo
// Parent state unrelated to child  Split state / localize
// Context triggers large re-renders    Split context providers
// Heavy component trees    Lazy load / virtualization

// You need to share data across deeply nested components.how do you approach it?
// 1. Prop Drilling
// •    Passing props from parent → child → grandchild repeatedly.
// •    Works for small trees or few nested levels.
// function Parent() {
//   const user = { name: "Sai" };
//   return <Child user={user} />;
// }

// function Child({ user }) {
//   return <GrandChild user={user} />;
// }

// function GrandChild({ user }) {
//   return <p>{user.name}</p>;
// }
// ✅ Simple for small apps
// ❌ Becomes tedious and error-prone with many levels → “prop drilling problem”
// ____________________________________
// 🔹 2. React Context API
// •    Avoids prop drilling by creating a global-ish data store accessible to all nested
// Example:
// const UserContext = React.createContext();

// function App() {
//   const user = { name: "Sai" };
//   return (
//     <UserContext.Provider value={user}>
//       <Parent />
//     </UserContext.Provider>
//   );
// }

// function GrandChild() {
//   const user = React.useContext(UserContext);
//   return <p>{user.name}</p>;
// }
// ✅ Good for theming, auth, language, global settings
```

```
// ⚠️ Frequent context updates re-render all consumers, so split context if needed
// ____________________________________________
// 🔹 3. State Management Libraries
// For large-scale apps, Context may become insufficient. Libraries provide more control,
// •    Redux / Redux Toolkit – centralized store, predictable state, middlewares for asyn
// •    Zustand – simple, minimal re-renders, modern alternative.
// •    MobX – observable state, auto-reactive components.
// Example with Redux:
// // Store
// const initialState = { user: { name: "Sai" } };
// // Component
// const name = useSelector(state => state.user.name);
// ✅ Works well for large apps, easy debugging and devtools integration
// ____________________________________________
// 🔹 4. Other Patterns
// 1.   Custom Hooks – encapsulate logic and state to share across multiple components.
// function useUser() {
//   const [user, setUser] = useState({ name: "Sai" });
//   return { user, setUser };
// }
// •    Can be combined with Context for global state.
// 2.   Render Props / HOCs – less common now, mostly replaced by hooks.
// ____________________________________________
// 🔹 5. Best Practices
// •    Use props for small, local data passing.
// •    Use Context for static or infrequently updated global data (theme, auth).
// •    Use state management libraries for frequently updated or complex data.
// •    Split contexts or stores to minimize unnecessary re-renders.

// How do you ensure accessibility (a11y) in your components?
// 1. Use Semantic HTML
// •    Use elements according to their meaning:
// o    <button> instead of <div> for clickable actions
// o    <header>, <main>, <footer> for layout
// o    <label> for form inputs
// •    Semantic HTML gives screen readers context automatically.
// <label htmlFor="email">Email</label>
// <input id="email" type="email" />
// ____________________________________________
// 🔹 2. ARIA Attributes
// •    ARIA (Accessible Rich Internet Applications) attributes provide extra info when se
// •    Common attributes:
// o    aria-label, aria-labelledby → label for elements
// o    aria-hidden → hide elements from screen readers
// o    role → define element role when semantic HTML isn't used
// <button aria-label="Close menu">X</button>
// ✅ Use ARIA only when necessary; prefer semantic HTML first.
// ____________________________________________
// 🔹 3. Keyboard Accessibility
// •    Ensure all interactive elements are focusable with Tab.
// •    Use proper keyboard events (Enter, Space) for actions.
// •    Avoid using div or span as buttons unless you handle keyboard interactions.
```

```
// <div
//   role="button"
//   tabIndex={0}
//   onKeyDown={(e) => e.key === 'Enter' && doSomething()}
//   onClick={doSomething}
// >
//   Click me
// </div>
// •    Better: just use <button>.
// ____________________________________________
// 🔹 4. Focus Management
// •    Manage focus for modals, dialogs, and dynamic content:
// o    Trap focus inside modals (react-focus-lock)
// o    Return focus to the triggering element when closing modal
// // Example: useRef to focus input on modal open
// const inputRef = useRef(null);
// useEffect(() => { inputRef.current.focus(); }, []);
// ____________________________________________
// 🔹 5. Color Contrast & Visual Cues
// •    Ensure text contrast ratio ≥ 4.5:1 for readability.
// •    Avoid color-only indicators; use icons or text alongside colors.
// ____________________________________________
// 🔹 6. Testing Accessibility
// •    Automated tools:
// o    eslint-plugin-jsx-a11y → linting rules for React JSX
// o    axe-core → accessibility testing
// •    Manual testing:
// o    Keyboard navigation
// o    Screen reader testing (VoiceOver, NVDA)
// npm install eslint-plugin-jsx-a11y --save-dev
// ____________________________________________
// 🔹 7. Best Practices in React
// •    Prefer controlled form components with <label> for accessibility.
// •    Avoid auto-focusing elements unnecessarily; it can confuse screen readers.
// •    Provide alt text for images and descriptive links.
// •    Use skip links (<a href="#main">Skip to content</a>) for keyboard users.

// You're asked to migrate a class-based component to a functional one with hooks.what are
// 1. Identify the Component Structure
// •    Take note of the class component’s:
// o    State (this.state)
// o    Lifecycle methods (componentDidMount, componentDidUpdate, componentWillUnmount)
// o    Event handlers
// o    Props usage
// Example class component:
// class Counter extends React.Component {
//   constructor(props) {
//     super(props);
//     this.state = { count: 0 };
//   }

//   componentDidMount() {
```

```
1285    //     console.log("Mounted");
1286    //   }
1287
1288    //   componentDidUpdate() {
1289    //     console.log("Updated");
1290    //   }
1291
1292    //   componentWillUnmount() {
1293    //     console.log("Unmounting");
1294    //   }
1295
1296    //   increment = () => {
1297    //     this.setState({ count: this.state.count + 1 });
1298    //   };
1299
1300    //   render() {
1301    //     return (
1302    //       <div>
1303    //         <p>{this.state.count}</p>
1304    //         <button onClick={this.increment}>Increment</button>
1305    //       </div>
1306    //     );
1307    //   }
1308    // }
1309    // ____________________________________________
1310    //  🔹 2. Convert to Functional Component
1311    // •    Remove the class keyword and render() method.
1312    // •    Create a function that takes props.
1313    // function Counter(props) {
1314    //   // logic goes here
1315    //   return (
1316    //     <div>
1317    //       {/* JSX */}
1318    //     </div>
1319    //   );
1320    // }
1321    // ____________________________________________
1322    //  🔹 3. Convert State to useState
1323    // •    Replace this.state and this.setState with useState.
1324    // const [count, setCount] = useState(0);
1325
1326    // const increment = () => setCount(count + 1);
1327    // ____________________________________________
1328    //  🔹 4. Replace Lifecycle Methods with useEffect
1329    // Class Lifecycle  Functional Hook Equivalent
1330    // componentDidMount    useEffect(() => { ... }, [])
1331    // componentDidUpdate   useEffect(() => { ... }, [dependencies])
1332    // componentWillUnmount useEffect(() => { return () => { ... } }, [])
1333    // Example:
1334    // useEffect(() => {
1335    //   console.log("Mounted or updated"); // combine mount/update logic
1336    //   return () => console.log("Unmounting"); // cleanup
```

```
// }, [count]); // dependency array
// ____________________________________________
// 🔹 5. Convert Event Handlers
// •   Remove this references.
// •   Use arrow functions or defined functions inside the component.
// const increment = () => setCount(count + 1);
// <button onClick={increment}>Increment</button>
// ____________________________________________
// 🔹 6. Props Handling
// •   this.props → directly use props argument.
// function Counter({ initialCount }) {
//   const [count, setCount] = useState(initialCount);
// }
// ____________________________________________
// 🔹 7. Test Component Behavior
// •   Verify:
// o   State updates correctly
// o   Lifecycle logic works (mounting, updating, cleanup)
// o   Event handlers function as expected
// ____________________________________________
// 🔹 8. Optional: Optimize with Hooks
// •   If needed, add:
// o   useMemo for expensive computations
// o   useCallback for memoized handlers
// o   useRef for DOM references
// ____________________________________________
// ✅ Final Functional Component Example
// function Counter({ initialCount = 0 }) {
//   const [count, setCount] = useState(initialCount);

//   useEffect(() => {
//     console.log("Mounted or updated");
//     return () => console.log("Unmounting");
//   }, [count]);

//   const increment = () => setCount(count + 1);

//   return (
//     <div>
//       <p>{count}</p>
//       <button onClick={increment}>Increment</button>
//     </div>
//   );
// }
// ____________________________________________
// Summary Steps:
// 1.   Convert class → function.
// 2.   Replace state with useState.
// 3.   Replace lifecycle methods with useEffect.
// 4.   Remove this references in props, state, and methods.
// 5.   Update event handlers.
// 6.   Test for correct behavior.
```

```
// 7.   Optionally optimize with useMemo, useCallback, or useRef.

// An API fetch needs to be cancelled if the user navigates away.how would you implement t
// 1. Using AbortController with fetch
// import { useEffect, useState } from "react";

// function UserList() {
//   const [users, setUsers] = useState([]);
//   const [loading, setLoading] = useState(true);

//   useEffect(() => {
//     const controller = new AbortController(); // create a controller
//     const signal = controller.signal;

//     async function fetchUsers() {
//       try {
//         const response = await fetch("https://api.example.com/users", { signal });
//         const data = await response.json();
//         setUsers(data);
//         setLoading(false);
//       } catch (error) {
//         if (error.name === "AbortError") {
//           console.log("Fetch aborted");
//         } else {
//           console.error("Fetch error:", error);
//         }
//       }
//     }

//     fetchUsers();

//     // Cleanup: cancel fetch if component unmounts
//     return () => controller.abort();
//   }, []); // empty dependency → runs once on mount

//   if (loading) return <p>Loading...</p>;
//   return (
//     <ul>
//       {users.map(user => <li key={user.id}>{user.name}</li>)}
//     </ul>
//   );
// }
// ____________________________________________
//  🔹 2. Key Points
// 1.   AbortController is a native JS API for canceling fetch requests.
// 2.   The signal option links the fetch to the controller.
// 3.   In the useEffect cleanup function, calling controller.abort() cancels the fetch if
// 4.   Always handle the "AbortError" to avoid logging unnecessary errors.
// ____________________________________________
//  🔹 3. With Axios
// If you use Axios instead of fetch, it has a CancelToken (Axios < v1) or AbortController
// import axios from "axios";
```

```
// import { useEffect, useState } from "react";

// function UserList() {
//   const [users, setUsers] = useState([]);
//   useEffect(() => {
//     const controller = new AbortController();

//     axios.get("https://api.example.com/users", { signal: controller.signal })
//       .then(res => setUsers(res.data))
//       .catch(err => {
//         if (axios.isCancel(err)) console.log("Request cancelled");
//         else console.error(err);
//       });

//     return () => controller.abort();
//   }, []);

//   return <div>{users.length} users loaded</div>;
// }
// ____________________________________
//  🔹 4. Best Practices
// •    Always cleanup async operations in useEffect.
// •    Avoid setting state on unmounted components.
// •    For multiple API calls, consider centralized abort controllers.
// •    If using libraries like React Query or SWR, they handle automatic cancellation for

// You need to make a component that is reused in multiple projects.how do you design it?
// 1. Identify the Core Functionality
// •    Determine what the component must do versus what can be customizable.
// •    Example: A Button component must handle clicks, display a label, and support diffe
// ____________________________________
//  🔹 2. Design a Clear API (Props)
// •    Expose props for customization instead of hardcoding values.
// •    Props can control:
// o    Content → children, label
// o    Styling → className, style, variant, size
// o    Behavior → onClick, disabled, loading
// •    Keep default values for common use cases.
// function Button({ label, onClick, size = "medium", variant = "primary", disabled = fals
//   return (
//     <button
//       onClick={onClick}
//       disabled={disabled}
//       className={`btn ${variant} ${size}`}
//     >
//       {label}
//     </button>
//   );
// }
// ____________________________________
//  🔹 3. Styling Flexibility
// •    Avoid hard-coded styles. Use:
```

```
// o    CSS Modules / Tailwind / styled-components
// o    Allow className or style overrides for custom styling.
// <button className={`btn ${variant} ${size} ${className}`} />
// •    Enables project-specific styling without changing the component code.
// ____________________________________________
// 🔹 4. Accessibility (a11y)
// •    Include semantic HTML and ARIA attributes by default.
// •    Example for a Button:
// o    Ensure keyboard accessibility
// o    Handle disabled and aria-label
// ____________________________________________
// 🔹 5. Handle Variability & Extensibility
// •    Make the component composable: support custom children or render props if needed.
// <Button>
//   <span>🚀 Launch</span>
// </Button>
// •    Use slots, render props, or children for advanced customization.
// ____________________________________________
// 🔹 6. Avoid Project-Specific Dependencies
// •    Component should be agnostic of a particular project.
// •    Don’t import project-specific state, context, or assets.
// •    Accept everything configurable via props.
// ____________________________________________
// 🔹 7. Testing
// •    Write unit tests for:
// o    Rendering with different props
// o    Event handling (onClick)
// o    Accessibility (aria-* attributes)
// ____________________________________________
// 🔹 8. Documentation
// •    Provide clear prop types and usage examples.
// •    Use:
// o    propTypes or TypeScript interfaces
// o    Storybook or a demo page for visualization
// interface ButtonProps {
//   label: string;
//   onClick: () => void;
//   size?: "small" | "medium" | "large";
//   variant?: "primary" | "secondary";
//   disabled?: boolean;
// }
// ____________________________________________
// 🔹 9. Packaging for Reuse
// •    If sharing across projects:
// o    Publish as an npm package
// o    Or use a monorepo (e.g., Turborepo, Nx)
// o    Ensure versioning, changelog, and dependencies are clear
// ____________________________________________
// 🔹 10. Best Practices Summary
// 1.   Keep API simple but flexible.
// 2.   Make styling customizable via className or theme props.
// 3.   Ensure accessibility by default.
```

```
// 4.   Keep it decoupled from specific projects.
// 5.   Support composition and advanced use cases.
// 6.   Write tests and documentation.
// 7.   Package for easy integration.

// How do you secure a React frontend in terms of handling tokens and sensitive info?
// 1. Don’t Store Sensitive Secrets in the Frontend
// •    Never hardcode API keys, secrets, or credentials in your React code.
// •    All sensitive logic should live on the server.
// •    Frontend can only store temporary tokens like JWT for authentication.
// ____________________________________________
// 🔹 2. Handling Tokens Securely
// a) Prefer HttpOnly Cookies over localStorage/sessionStorage
// •    HttpOnly cookies cannot be accessed via JavaScript → protects against XSS attacks.
// •    Example: backend sets cookie with Set-Cookie: HttpOnly; Secure; SameSite=Strict.
// b) If Using localStorage / sessionStorage
// •    Only for short-lived tokens or non-critical data.
// •    Always validate tokens on the backend.
// c) Token Expiration & Refresh
// •    Use short-lived access tokens + refresh tokens.
// •    Automatically refresh tokens via silent API calls, not manual input.
// ____________________________________________
// 🔹 3. Protect Against XSS (Cross-Site Scripting)
// •    Sanitize user input before rendering.
// •    Avoid dangerouslySetInnerHTML unless necessary and sanitized.
// •    Use libraries like DOMPurify if rendering HTML from external sources.
// import DOMPurify from 'dompurify';

// <p dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(userInput) }} />
// ____________________________________________
// 🔹 4. Secure API Requests
// •    Use HTTPS for all API requests.
// •    Add CORS restrictions on the backend.
// •    Don’t rely on frontend-only security; always validate tokens server-side.
// fetch('/api/data', {
//   method: 'GET',
//   headers: {
//     'Authorization': `Bearer ${accessToken}`,
//   },
// });
// ____________________________________________
// 🔹 5. Avoid Storing Sensitive Data in Memory for Long
// •    Keep tokens in state or memory while needed, and clear on logout.
// •    Don’t leave sensitive info in global variables or browser storage longer than nece
// ____________________________________________
// 🔹 6. Secure Routing
// •    Protect routes using authentication guards.
// •    Example with React Router:
// function PrivateRoute({ children }) {
//   const isAuthenticated = Boolean(localStorage.getItem("token"));
//   return isAuthenticated ? children : <Navigate to="/login" />;
// }
```

```
// •    Always verify auth on backend, not just frontend.
// ____________________________________________
// 🔹 7. Miscellaneous Best Practices
// 1.   Content Security Policy (CSP) – prevents malicious scripts from executing.
// 2.   Subresource Integrity (SRI) – validate external scripts.
// 3.   Environment Variables – use .env for config, never commit secrets.
// 4.   Secure dependencies – keep packages up to date and scan for vulnerabilities.
// ____________________________________________
// 🔹 8. Summary Table
// Risk / Requirement   Best Practice
// Token storage    HttpOnly cookies preferred, avoid localStorage for sensitive data
// XSS protection   Sanitize input, avoid dangerouslySetInnerHTML
// API communication    HTTPS + backend validation + CORS
// Routing / access control Frontend guards + backend validation
// Secrets  Never hardcode, use environment variables
// Token lifecycle  Short-lived + refresh tokens
```

```
// // Machine Coding
// // *Build a counter app with increment, decrement and reset buttons.
// import React, { useState } from "react";

// function CounterApp() {
//   const [count, setCount] = useState(0);

//   const increment = () => setCount(prevCount => prevCount + 1);
//   const decrement = () => setCount(prevCount => prevCount >0 ? prevCount-1:0);
//   const reset = () => setCount(0);

//   return (
//     <div style={styles.container}>
//       <h1>Counter App</h1>
//       <p style={styles.count}>{count}</p>
//       <div style={styles.buttonContainer}>
//         <button onClick={increment} style={styles.button}>Increment</button>

//         <button disable={count===0} onClick={decrement} style={styles.button}>Decrement
//         <button onClick={reset} style={styles.button}>Reset</button>
//       </div>
//     </div>
//   );
// }

// // Inline styles
// const styles = {
```

```
1649  //   container: {
1650  //     display: "flex",
1651  //     flexDirection: "column",
1652  //     alignItems: "center",
1653  //     marginTop: "50px",
1654  //     fontFamily: "Arial, sans-serif",
1655  //   },
1656  //   count: {
1657  //     fontSize: "48px",
1658  //     margin: "20px 0",
1659  //   },
1660  //   buttonContainer: {
1661  //     display: "flex",
1662  //     gap: "10px",
1663  //   },
1664  //   button: {
1665  //     padding: "10px 20px",
1666  //     fontSize: "16px",
1667  //     cursor: "pointer",
1668  //   },
1669  // };
1670
1671  // export default CounterApp;
1672  // {/* -------------------------------- */}
1673  //  {/* *create a crud operation - name, gender option, state, city dropdown,
1674  //  pincode validation, email validation, mobile number validation, Choose
1675  //   Department dropdown, salary currency field, and end of form confirm
1676  //    box */}
1677
1678  // import React, { useState, useEffect } from "react";
1679
1680  // // Sample states and cities
1681  // const stateCityData = {
1682  //   California: ["Los Angeles", "San Francisco", "San Diego"],
1683  //   Texas: ["Houston", "Dallas", "Austin"],
1684  //   Florida: ["Miami", "Orlando", "Tampa"],
1685  // };
1686
1687  // // Sample departments
1688  // const departments = ["Engineering", "HR", "Marketing", "Sales"];
1689
1690  // function CrudForm() {
1691  //   const initialFormState = {
1692  //     id: null,
1693  //     name: "",
1694  //     gender: "",
1695  //     state: "",
1696  //     city: "",
1697  //     pincode: "",
1698  //     email: "",
1699  //     mobile: "",
1700  //     department: "",
```

```
1701  //     salary: "",
1702  //     confirm: false,
1703  //   };
1704
1705  //   const [formData, setFormData] = useState(initialFormState);
1706  //   const [entries, setEntries] = useState([]);
1707  //   const [cities, setCities] = useState([]);
1708  //   const [editId, setEditId] = useState(null);
1709  //   const [errors, setErrors] = useState({});
1710
1711  //   // Update cities when state changes
1712  //   useEffect(() => {
1713  //     if (formData.state) setCities(stateCityData[formData.state] || []);
1714  //     else setCities([]);
1715  //     setFormData({ ...formData, city: "" }); // reset city
1716  //   }, [formData.state]);
1717
1718  //   // Handle input change
1719  //   const handleChange = (e) => {
1720  //     const { name, value, type, checked } = e.target;
1721  //     setFormData({ ...formData, [name]: type === "checkbox" ? checked : value });
1722  //   };
1723
1724  //   // Validate form
1725  //   const validate = () => {
1726  //     const newErrors = {};
1727  //     if (!formData.name) newErrors.name = "Name is required";
1728  //     if (!formData.gender) newErrors.gender = "Gender is required";
1729  //     if (!formData.state) newErrors.state = "State is required";
1730  //     if (!formData.city) newErrors.city = "City is required";
1731  //     if (!/^\d{5,6}$/.test(formData.pincode)) newErrors.pincode = "Invalid pincode";
1732  //     if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) newErrors.email = "Invalid
1733  //     if (!/^\d{10}$/.test(formData.mobile)) newErrors.mobile = "Invalid mobile number";
1734  //     if (!formData.department) newErrors.department = "Select a department";
1735  //     if (!formData.salary) newErrors.salary = "Salary is required";
1736  //     if (!formData.confirm) newErrors.confirm = "Please confirm the form";
1737  //     setErrors(newErrors);
1738  //     return Object.keys(newErrors).length === 0;
1739  //   };
1740
1741  //   // Handle submit
1742  //   const handleSubmit = (e) => {
1743  //     e.preventDefault();
1744  //     if (!validate()) return;
1745
1746  //     if (editId !== null) {
1747  //       setEntries(
1748  //         entries.map((entry) => (entry.id === editId ? { ...formData, id: editId } : ent
1749  //       );
1750  //       setEditId(null);
1751  //     } else {
1752  //       setEntries([...entries, { ...formData, id: Date.now() }]);
```

```
1753  //     }
1754  //     setFormData(initialFormState);
1755  //   };
1756  //
1757  //   // Handle edit
1758  //   const handleEdit = (id) => {
1759  //     const entry = entries.find((e) => e.id === id);
1760  //     setFormData(entry);
1761  //     setEditId(id);
1762  //   };
1763  //
1764  //   // Handle delete
1765  //   const handleDelete = (id) => setEntries(entries.filter((e) => e.id !== id));
1766  //
1767  //   return (
1768  //     <div style={styles.container}>
1769  //       <h2>Employee Form</h2>
1770  //       <form onSubmit={handleSubmit} style={styles.form}>
1771  //         <label>
1772  //           Name:
1773  //           <input type="text" name="name" value={formData.name} onChange={handleChange}
1774  //           {errors.name && <span style={styles.error}>{errors.name}</span>}
1775  //         </label>
1776  //
1777  //         <label>
1778  //           Gender:
1779  //           <select name="gender" value={formData.gender} onChange={handleChange}>
1780  //             <option value="">Select Gender</option>
1781  //             <option value="Male">Male</option>
1782  //             <option value="Female">Female</option>
1783  //           </select>
1784  //           {errors.gender && <span style={styles.error}>{errors.gender}</span>}
1785  //         </label>
1786  //
1787  //         <label>
1788  //           State:
1789  //           <select name="state" value={formData.state} onChange={handleChange}>
1790  //             <option value="">Select State</option>
1791  //             {Object.keys(stateCityData).map((state) => (
1792  //               <option key={state} value={state}>{state}</option>
1793  //             ))}
1794  //           </select>
1795  //           {errors.state && <span style={styles.error}>{errors.state}</span>}
1796  //         </label>
1797  //
1798  //         <label>
1799  //           City:
1800  //           <select name="city" value={formData.city} onChange={handleChange}>
1801  //             <option value="">Select City</option>
1802  //             {cities.map((city) => (
1803  //               <option key={city} value={city}>{city}</option>
1804  //             ))}
```

```
1805  //           </select>
1806  //           {errors.city && <span style={styles.error}>{errors.city}</span>}
1807  //         </label>
1808
1809  //         <label>
1810  //           Pincode:
1811  //           <input type="text" name="pincode" value={formData.pincode} onChange={handleCh
1812  //           {errors.pincode && <span style={styles.error}>{errors.pincode}</span>}
1813  //         </label>
1814
1815  //         <label>
1816  //           Email:
1817  //           <input type="email" name="email" value={formData.email} onChange={handleChang
1818  //           {errors.email && <span style={styles.error}>{errors.email}</span>}
1819  //         </label>
1820
1821  //         <label>
1822  //           Mobile:
1823  //           <input type="text" name="mobile" value={formData.mobile} onChange={handleChan
1824  //           {errors.mobile && <span style={styles.error}>{errors.mobile}</span>}
1825  //         </label>
1826
1827  //         <label>
1828  //           Department:
1829  //           <select name="department" value={formData.department} onChange={handleChange}
1830  //             <option value="">Select Department</option>
1831  //             {departments.map((dept) => (
1832  //               <option key={dept} value={dept}>{dept}</option>
1833  //             ))}
1834  //           </select>
1835  //           {errors.department && <span style={styles.error}>{errors.department}</span>}
1836  //         </label>
1837
1838  //         <label>
1839  //           Salary (USD):
1840  //           <input type="number" name="salary" value={formData.salary} onChange={handleCh
1841  //           {errors.salary && <span style={styles.error}>{errors.salary}</span>}
1842  //         </label>
1843
1844  //         <label>
1845  //           <input type="checkbox" name="confirm" checked={formData.confirm} onChange={ha
1846  //           Confirm all information
1847  //           {errors.confirm && <span style={styles.error}>{errors.confirm}</span>}
1848  //         </label>
1849
1850  //         <button type="submit">{editId !== null ? "Update" : "Submit"}</button>
1851  //       </form>
1852
1853  //       <h3>Employee List</h3>
1854  //       {entries.length === 0 ? <p>No entries yet</p> : (
1855  //         <table style={styles.table}>
1856  //           <thead>
```

```
1857  //             <tr>
1858  //               <th>Name</th>
1859  //               <th>Gender</th>
1860  //               <th>State</th>
1861  //               <th>City</th>
1862  //               <th>Pincode</th>
1863  //               <th>Email</th>
1864  //               <th>Mobile</th>
1865  //               <th>Department</th>
1866  //               <th>Salary</th>
1867  //               <th>Actions</th>
1868  //             </tr>
1869  //           </thead>
1870  //           <tbody>
1871  //             {entries.map((e) => (
1872  //               <tr key={e.id}>
1873  //                 <td>{e.name}</td>
1874  //                 <td>{e.gender}</td>
1875  //                 <td>{e.state}</td>
1876  //                 <td>{e.city}</td>
1877  //                 <td>{e.pincode}</td>
1878  //                 <td>{e.email}</td>
1879  //                 <td>{e.mobile}</td>
1880  //                 <td>{e.department}</td>
1881  //                 <td>${e.salary}</td>
1882  //                 <td>
1883  //                   <button onClick={() => handleEdit(e.id)}>Edit</button>
1884  //                   <button onClick={() => handleDelete(e.id)}>Delete</button>
1885  //                 </td>
1886  //               </tr>
1887  //             ))}
1888  //           </tbody>
1889  //         </table>
1890  //       )}
1891  //     </div>
1892  //   );
1893  // }
1894
1895  // // Simple styles
1896  // const styles = {
1897  //   container: { margin: "20px", fontFamily: "Arial, sans-serif" },
1898  //   form: { display: "flex", flexDirection: "column", gap: "10px", maxWidth: "400px" },
1899  //   error: { color: "red", fontSize: "12px", marginLeft: "5px" },
1900  //   table: { borderCollapse: "collapse", width: "100%", marginTop: "20px" },
1901  //   tableCell: { border: "1px solid #ccc", padding: "5px" },
1902  // };
1903
1904  // export default CrudForm;
1905
1906
1907
1908
```

```
1909
1910    // // *Build a toggle switch (on/off with state).
1911    // import React, { useState, useEffect } from "react";
1912
1913    // function ToggleSwitch() {
1914    //   const [isOn, setIsOn] = useState(false);
1915
1916    //   const handleToggle = () => setIsOn(prev => !prev);
1917
1918    //   // Change body background based on toggle state
1919    //   useEffect(() => {
1920    //     document.body.style.backgroundColor = isOn ? "#d4f4dd" : "#f4d4d4"; // greenish for
1921    //   }, [isOn]);
1922
1923    //   return (
1924    //     <div style={styles.container}>
1925    //       <span style={styles.label}>{isOn ? "ON" : "OFF"}</span>
1926    //       <div
1927    //         onClick={handleToggle}
1928    //         style={{
1929    //           ...styles.switch,
1930    //           backgroundColor: isOn ? "#4caf50" : "#ccc",
1931    //           justifyContent: isOn ? "flex-end" : "flex-start"
1932    //         }}
1933    //       >
1934    //         <div style={styles.knob}></div>
1935    //       </div>
1936    //     </div>
1937    //   );
1938    // }
1939
1940    // // Inline styles
1941    // const styles = {
1942    //   container: {
1943    //     display: "flex",
1944    //     alignItems: "center",
1945    //     gap: "10px",
1946    //     fontFamily: "Arial, sans-serif",
1947    //     marginTop: "50px",
1948    //   },
1949    //   label: {
1950    //     fontSize: "18px",
1951    //     fontWeight: "bold",
1952    //   },
1953    //   switch: {
1954    //     width: "60px",
1955    //     height: "30px",
1956    //     borderRadius: "30px",
1957    //     display: "flex",
1958    //     alignItems: "center",
1959    //     padding: "3px",
1960    //     cursor: "pointer",
```

```
1961    //     transition: "background-color 0.3s, justify-content 0.3s",
1962    //   },
1963    //   knob: {
1964    //     width: "24px",
1965    //     height: "24px",
1966    //     borderRadius: "50%",
1967    //     backgroundColor: "#fff",
1968    //     transition: "0.3s",
1969    //   },
1970    // };
1971
1972    // export default ToggleSwitch;
1973
1974    // // *Create a searchable list that filters items based on user input.
1975    // import React, { useState, useEffect } from "react";
1976
1977    // function SearchableApiList() {
1978    //   const [items, setItems] = useState([]);
1979    //   const [searchTerm, setSearchTerm] = useState("");
1980    //   const [loading, setLoading] = useState(true);
1981    //   const [error, setError] = useState(null);
1982
1983    //   // Fetch data from API
1984    //   useEffect(() => {
1985    //     const fetchData = async () => {
1986    //       try {
1987    //         const response = await fetch("https://jsonplaceholder.typicode.com/users");
1988    //         if (!response.ok) throw new Error("Failed to fetch data");
1989    //         const data = await response.json();
1990    //         setItems(data);
1991    //       } catch (err) {
1992    //         setError(err.message);
1993    //       } finally {
1994    //         setLoading(false);
1995    //       }
1996    //     };
1997
1998    //     fetchData();
1999    //   }, []);
2000
2001    //   // Filter items by name based on search term
2002    //   const filteredItems = items.filter((item) =>
2003    //     item.name.toLowerCase().includes(searchTerm.toLowerCase())
2004    //   );
2005
2006    //   if (loading) return <p>Loading...</p>;
2007    //   if (error) return <p>Error: {error}</p>;
2008
2009    //   return (
2010    //     <div style={styles.container}>
2011    //       <h2>Search Users</h2>
2012    //       <input
```

```
2013  //         type="text"
2014  //         placeholder="Search by name..."
2015  //         value={searchTerm}
2016  //         onChange={(e) => setSearchTerm(e.target.value)}
2017  //         style={styles.input}
2018  //       />
2019
2020  //       <ul style={styles.list}>
2021  //         {filteredItems.length > 0 ? (
2022  //           filteredItems.map((item) => (
2023  //             <li key={item.id}>
2024  //               <strong>{item.name}</strong> - {item.email}
2025  //             </li>
2026  //           ))
2027  //         ) : (
2028  //           <li>No users found</li>
2029  //         )}
2030  //       </ul>
2031  //     </div>
2032  //   );
2033  // }
2034
2035  // // Inline styles
2036  // const styles = {
2037  //   container: { margin: "20px", fontFamily: "Arial, sans-serif" },
2038  //   input: { padding: "8px", width: "250px", marginBottom: "10px" },
2039  //   list: { listStyleType: "none", paddingLeft: 0 },
2040  // };
2041
2042  // export default SearchableApiList;
2043
2044  // // *Implement a tabbed UI using conditional rendering.
2045  // import React, { useState } from "react";
2046
2047  // function TabbedUI() {
2048  //   const [activeTab, setActiveTab] = useState("home");
2049
2050  //   return (
2051  //     <div style={styles.container}>
2052  //       <h2>Tabbed UI Example</h2>
2053
2054  //       {/* Tab Buttons */}
2055  //       <div style={styles.tabContainer}>
2056  //         <button
2057  //           style={activeTab === "home" ? styles.activeTab : styles.tab}
2058  //           onClick={() => setActiveTab("home")}
2059  //         >
2060  //           Home
2061  //         </button>
2062  //         <button
2063  //           style={activeTab === "profile" ? styles.activeTab : styles.tab}
2064  //           onClick={() => setActiveTab("profile")}
```

```
//         >
//           Profile
//         </button>
//         <button
//           style={activeTab === "settings" ? styles.activeTab : styles.tab}
//           onClick={() => setActiveTab("settings")}
//         >
//           Settings
//         </button>
//       </div>

//       {/* Tab Content */}
//       <div style={styles.content}>
//         {activeTab === "home" && <p>Welcome to the Home tab!</p>}
//         {activeTab === "profile" && <p>This is your Profile information.</p>}
//         {activeTab === "settings" && <p>Adjust your Settings here.</p>}
//       </div>
//     </div>
//   );
// }

// // Inline styles
// const styles = {
//   container: { margin: "20px", fontFamily: "Arial, sans-serif" },
//   tabContainer: { display: "flex", gap: "10px", marginBottom: "20px" },
//   tab: {
//     padding: "10px 20px",
//     cursor: "pointer",
//     backgroundColor: "#eee",
//     border: "none",
//     borderRadius: "5px",
//   },
//   activeTab: {
//     padding: "10px 20px",
//     cursor: "pointer",
//     backgroundColor: "#4caf50",
//     color: "#fff",
//     border: "none",
//     borderRadius: "5px",
//   },
//   content: { padding: "20px", border: "1px solid #ccc", borderRadius: "5px" },
// };

// export default TabbedUI;

// // *Build a form with validation (e.g., email, password).
// import React, { useState } from "react";

// function LoginForm() {
//   const [formData, setFormData] = useState({ email: "", password: "" });
//   const [errors, setErrors] = useState({});
//   const [showPassword, setShowPassword] = useState(false); // toggle state
```

```
2117
2118    //   const handleChange = (e) => {
2119    //     const { name, value } = e.target;
2120    //     setFormData({ ...formData, [name]: value });
2121    //   };
2122
2123    //   const validate = () => {
2124    //     const newErrors = {};
2125    //     if (!formData.email) {
2126    //       newErrors.email = "Email is required";
2127    //     } else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) {
2128    //       newErrors.email = "Invalid email address";
2129    //     }
2130
2131    //     if (!formData.password) {
2132    //       newErrors.password = "Password is required";
2133    //     } else if (formData.password.length < 6) {
2134    //       newErrors.password = "Password must be at least 6 characters";
2135    //     }
2136
2137    //     setErrors(newErrors);
2138    //     return Object.keys(newErrors).length === 0;
2139    //   };
2140
2141    //   const handleSubmit = (e) => {
2142    //     e.preventDefault();
2143    //     if (validate()) {
2144    //       alert(`Login successful!\nEmail: ${formData.email}`);
2145    //       setFormData({ email: "", password: "" });
2146    //     }
2147    //   };
2148
2149    //   return (
2150    //     <div style={styles.container}>
2151    //       <h2>Login Form</h2>
2152    //       <form onSubmit={handleSubmit} style={styles.form}>
2153    //         <label>
2154    //           Email:
2155    //           <input
2156    //             type="email"
2157    //             name="email"
2158    //             value={formData.email}
2159    //             onChange={handleChange}
2160    //             style={styles.input}
2161    //           />
2162    //           {errors.email && <span style={styles.error}>{errors.email}</span>}
2163    //         </label>
2164
2165    //         <label style={{ position: "relative" }}>
2166    //           Password:
2167    //           <input
2168    //             type={showPassword ? "text" : "password"}
```

```
//             name="password"
//             value={formData.password}
//             onChange={handleChange}
//             style={{ ...styles.input, paddingRight: "30px" }}
//           />
//           <span
//             onClick={() => setShowPassword(prev => !prev)}
//             style={styles.eyeIcon}
//           >
//             {showPassword ? "👁️" : "🙈"}
//           </span>
//           {errors.password && <span style={styles.error}>{errors.password}</span>}
//         </label>

//         <button type="submit" style={styles.button}>Login</button>
//       </form>
//     </div>
//   );
// }

// // Inline styles
// const styles = {
//   container: { margin: "20px", fontFamily: "Arial, sans-serif" },
//   form: { display: "flex", flexDirection: "column", gap: "15px", maxWidth: "300px" },
//   input: { padding: "8px", marginTop: "5px", width: "100%" },
//   button: { padding: "10px", backgroundColor: "#4caf50", color: "#fff", border: "none",
//   error: { color: "red", fontSize: "12px" },
//   eyeIcon: {
//     position: "absolute",
//     right: "10px",
//     top: "35px",
//     cursor: "pointer",
//     fontSize: "18px",
//     userSelect: "none",
//   },
// };

// export default LoginForm;

// // *Build a star rating component with hover and click.
// import React, { useState } from "react";

// function StarRating({ totalStars = 5 }) {
//   const [rating, setRating] = useState(0);       // selected rating
//   const [hover, setHover] = useState(0);         // hover rating

//   return (
//     <div style={styles.container}>
//       <h2>Star Rating</h2>
//       <div style={styles.stars}>
//         {Array.from({ length: totalStars }, (_, index) => {
//           const starValue = index + 1;
```

```
2221  //           return (
2222  //             <span
2223  //               key={index}
2224  //               style={{
2225  //                 ...styles.star,
2226  //                 color: starValue <= (hover || rating) ? "#ffc107" : "#ccc",
2227  //               }}
2228  //               onClick={() => setRating(starValue)}
2229  //               onMouseEnter={() => setHover(starValue)}
2230  //               onMouseLeave={() => setHover(0)}
2231  //             >
2232  //               ★
2233  //             </span>
2234  //           );
2235  //         })}
2236  //       </div>
2237  //       <p>Your Rating: {rating} / {totalStars}</p>
2238  //     </div>
2239  //   );
2240  // }
2241
2242  // // Inline styles
2243  // const styles = {
2244  //   container: { margin: "20px", fontFamily: "Arial, sans-serif" },
2245  //   stars: { display: "flex", gap: "5px", fontSize: "40px", cursor: "pointer" },
2246  //   star: { transition: "color 0.2s" },
2247  // };
2248
2249  // export default StarRating;
2250
2251  // // *Build a custom dropdown using keyboard navigation.
2252  // // *Create a reusable modal component with backdrop and escape handling.
2253  // model.jsx
2254  // import React, { useState } from "react";
2255  // import Modal from "./Modal";
2256
2257  // function App() {
2258  //   const [isModalOpen, setIsModalOpen] = useState(false);
2259
2260  //   return (
2261  //     <div style={{ padding: "20px" }}>
2262  //       <h2>Reusable Modal Example</h2>
2263  //       <button
2264  //         onClick={() => setIsModalOpen(true)}
2265  //         style={{ padding: "10px 20px", cursor: "pointer" }}
2266  //       >
2267  //         Open Modal
2268  //       </button>
2269
2270  //       <Modal isOpen={isModalOpen} onClose={() => setIsModalOpen(false)}>
2271  //         <h3>Modal Content</h3>
2272  //         <p>This is a reusable modal component with backdrop and escape handling.</p>
```

```
//       </Modal>
//     </div>
//   );
// }

// export default App;

// App.jsx
// import React, { useEffect } from "react";

// function Modal({ isOpen, onClose, children }) {
//   // Close on Escape key
//   useEffect(() => {
//     const handleKeyDown = (e) => {
//       if (e.key === "Escape") {
//         onClose();
//       }
//     };
//     if (isOpen) {
//       document.addEventListener("keydown", handleKeyDown);
//     }
//     return () => document.removeEventListener("keydown", handleKeyDown);
//   }, [isOpen, onClose]);

//   if (!isOpen) return null;

//   return (
//     <div style={styles.backdrop} onClick={onClose}>
//       <div
//         style={styles.modal}
//         onClick={(e) => e.stopPropagation()} // prevent closing when clicking inside mo
//       >
//         {children}
//         <button onClick={onClose} style={styles.closeButton}>
//           Close
//         </button>
//       </div>
//     </div>
//   );
// }

// // Styles
// const styles = {
//   backdrop: {
//     position: "fixed",
//     top: 0,
//     left: 0,
//     right: 0,
//     bottom: 0,
//     backgroundColor: "rgba(0,0,0,0.5)",
//     display: "flex",
//     justifyContent: "center",
```

```
2325  //     alignItems: "center",
2326  //     zIndex: 1000,
2327  //   },
2328  //   modal: {
2329  //     backgroundColor: "#fff",
2330  //     padding: "20px",
2331  //     borderRadius: "8px",
2332  //     minWidth: "300px",
2333  //     maxWidth: "500px",
2334  //     boxShadow: "0 2px 10px rgba(0,0,0,0.3)",
2335  //     position: "relative",
2336  //   },
2337  //   closeButton: {
2338  //     marginTop: "20px",
2339  //     padding: "8px 16px",
2340  //     backgroundColor: "#4caf50",
2341  //     color: "#fff",
2342  //     border: "none",
2343  //     cursor: "pointer",
2344  //     borderRadius: "4px",
2345  //   },
2346  // };
2347
2348  // export default Modal;
2349
2350
2351
2352  // // *Implement infinite scroll or pagination using an API.
2353  // import React, { useState, useEffect, useRef, useCallback } from "react";
2354
2355  // function InfiniteScrollList() {
2356  //   const [posts, setPosts] = useState([]);
2357  //   const [page, setPage] = useState(1);
2358  //   const [hasMore, setHasMore] = useState(true);
2359  //   const [loading, setLoading] = useState(false);
2360
2361  //   const observer = useRef();
2362
2363  //   // Fetch posts from API
2364  //   const fetchPosts = async (page) => {
2365  //     setLoading(true);
2366  //     try {
2367  //       const res = await fetch(`https://jsonplaceholder.typicode.com/posts?_limit=10&_pa
2368  //       const data = await res.json();
2369  //       if (data.length === 0) setHasMore(false);
2370  //       setPosts((prev) => [...prev, ...data]);
2371  //     } catch (err) {
2372  //       console.error(err);
2373  //     } finally {
2374  //       setLoading(false);
2375  //     }
2376  //   };
```

```
2377
2378    //   useEffect(() => {
2379    //     fetchPosts(page);
2380    //   }, [page]);
2381
2382    //   // Observer for last element
2383    //   const lastPostRef = useCallback(
2384    //     (node) => {
2385    //       if (loading) return;
2386    //       if (observer.current) observer.current.disconnect();
2387
2388    //       observer.current = new IntersectionObserver((entries) => {
2389    //         if (entries[0].isIntersecting && hasMore) {
2390    //           setPage((prev) => prev + 1);
2391    //         }
2392    //       });
2393
2394    //       if (node) observer.current.observe(node);
2395    //     },
2396    //     [loading, hasMore]
2397    //   );
2398
2399    //   return (
2400    //     <div style={styles.container}>
2401    //       <h2>Infinite Scroll Posts</h2>
2402    //       <ul style={styles.list}>
2403    //         {posts.map((post, index) => {
2404    //           if (posts.length === index + 1) {
2405    //             return (
2406    //               <li ref={lastPostRef} key={post.id} style={styles.item}>
2407    //                 <strong>{post.id}. {post.title}</strong>
2408    //               </li>
2409    //             );
2410    //           } else {
2411    //             return (
2412    //               <li key={post.id} style={styles.item}>
2413    //                 <strong>{post.id}. {post.title}</strong>
2414    //               </li>
2415    //             );
2416    //           }
2417    //         })}
2418    //       </ul>
2419    //       {loading && <p>Loading...</p>}
2420    //       {!hasMore && <p>No more posts</p>}
2421    //     </div>
2422    //   );
2423    // }
2424
2425    // // Inline styles
2426    // const styles = {
2427    //   container: { margin: "20px", fontFamily: "Arial, sans-serif" },
2428    //   list: { listStyle: "none", padding: 0 },
```

```
2429    //   item: { padding: "10px", borderBottom: "1px solid #ccc" },
2430    // };
2431
2432    // export default InfiniteScrollList;
2433
2434    // // *Drag-and-drop functionality with custom hooks.
```